

OpoFit — Documentación técnica personal del proyecto

Autor: Rafael Javier Luengo Barreno

Qué es este documento: mi guía larga para **entender y, si hiciera falta, volver a montar** el sistema. Lo escribo en primera persona porque son **mis decisiones**, mis límites de tiempo y mi forma de razonar el código —no un manual anónimo. Aquí no vas a encontrar contadores de palabras ni estimaciones de páginas; solo explicación, referencias a rutas reales del repositorio y ejemplos cuando aclaran.

1. Por qué documento así (y para quién)

Este texto lo escribo principalmente para **mí**: para dentro de un año, cuando ya no recuerde por qué metí una regla en el servidor o por qué una pantalla llama a un endpoint y no a otro. Si le sirve a alguien más (tribunal, compañero, yo del futuro), mejor que mejor.

He evitado dos extremos: el **manual de API** que solo lista URLs sin contexto, y el **tutorial genérico** que no nombra archivos. Lo que busco es el punto medio: **historia + mapa del código**.

También quiero que se note **quién está detrás**: cuando digo “decidí” o “me frustró”, no es relleno; es la forma en que recuerdo el proceso. Un proyecto de estas características no sale de copiar un único tutorial de tres horas: sale de encadenar errores, soluciones parciales y refactors cuando entendí mejor el dominio.

El tono es técnico pero hablado: si buscas una tabla de endpoints con códigos HTTP y nada más, el código ya tiene las rutas; si buscas **por qué** el controlador de rutinas ignora el `userId` de la URL, aquí lo tienes explicado sin pedirte que adivines la intención.

Por último: **no incluyo** en este documento metadatos de “cuántas páginas ocupa impreso” ni contadores automáticos de palabras. Prefiero que el valor esté en el contenido útil, no en una cifra que envejece mal cada vez que añado un párrafo.

2. Idea del producto en una frase (y lo que no es)

OpoFit es una aplicación para **opositores con pruebas físicas**: centraliza información de convocatoria, propone rutinas de entrenamiento en función de perfil y oposición, permite rutinas personalizadas y guarda historial de sesiones y resultados por ejercicio para poder ver evolución.

No es un sustituto médico, un plan nutricional completo ni una red social deportiva. **Es** una herramienta de organización y seguimiento con datos persistidos en servidor.

En la práctica, cuando alguien me preguntaba “¿y qué hace tu app?”, respondía con una imagen: **un cuaderno de entrenos que no se pierde**, que además entiende que no todas las oposiciones son iguales. Esa imagen me ayudó a no desviarme hacia funciones que suenan bien en una lista de features pero no encajan en el tiempo de un TFG: por ejemplo, comparaciones sociales entre usuarios o rankings públicos. Los descarté temprano porque añaden complejidad legal, moderación y privacidad que no podía asumir con la profundidad debida.

También quiero dejar claro qué entiendo por “**rutina**” en este proyecto: no es un PDF estático descargable, sino un conjunto estructurado de ejercicios con parámetros (series, repeticiones, descansos) que el sistema puede mostrar paso a paso y sobre el cual puede registrar **resultados**

cuantificables. Esa distinción es la que justifica tablas de detalle separadas de la cabecera de rutina.

Por último, el adjetivo “oficial” en rutinas oficiales no significa que el cuerpo policial vaya a certificar mi base de datos; significa que las combinaciones nivel/género/enfoque/oposición están **precargadas** para alinearse con la lógica de preparación que modelé, frente a las rutinas libres donde el usuario es el autor.

Si al leer esto tienes la sensación de que **podrías volver a levantar** el sistema con tiempo y paciencia, he cumplido mi objetivo. Si en algún punto echas en falta un detalle microscópico (nombre exacto de un campo en un JSON concreto), ve al modelo Kotlin o al controlador: lo actualizo en código antes que en un PDF estático.

3. Arquitectura general que defendí

Separé **tres capas lógicas**:

1. **Cliente Android (Kotlin, Jetpack Compose)** — interfaz, estado de pantalla, llamadas HTTP.
2. **API REST (Node.js, Express)** — autenticación, reglas de negocio, acceso a datos.
3. **MySQL** — persistencia relacional con integridad referencial.

El móvil **nunca** toca la base de datos directamente. Todo pasa por HTTP; así el contrato es explícito y puedo cambiar detalles del SQL sin reescribir la app entera si mantengo las respuestas.

3.1. Por qué no microservicios (al menos aquí)

Un TFG con un solo desarrollador rara vez se beneficia de partir el backend en cinco despliegues distintos: multiplica configuración, observabilidad y puntos de fallo. Opté por un **monolito modular** dentro de Express: carpetas por dominio, pero un solo proceso. Si el proyecto creciera de verdad, podría extraer servicios (por ejemplo un “news service” para RSS) sin reescribir desde cero: la frontera natural ya está en archivos separados.

3.2. Sincronía frente a asincronía en Node

Express maneja peticiones de forma concurrente con su modelo de event loop. Mis operaciones a MySQL son `async/await` para no bloquear innecesariamente. Cuando abro transacciones, serializo el trabajo en una conexión dedicada hasta `commit/rollback` —no mezclo esa conexión con queries paralelas que no correspondan a la misma transacción.

3.3. Por qué REST y no GraphQL

GraphQL brilla con clientes heterogéneos y consultas flexibles; también añade capa de esquema y complejidad. Para una app móvil con un conjunto acotado de pantallas, REST con endpoints claros me permitió avanzar más rápido y depurar con herramientas simples. Si el día de mañana el cliente pide “solo dos campos” de rutinas gigantes, podría plantear optimizaciones; no fue el cuello de botella del TFG.

3.4. Estado en el servidor

El JWT es **stateless** en el sentido de que no guardo sesiones en memoria del servidor por usuario. El estado vive en cliente (token) y en base de datos (datos). Trade-off: revocar tokens antes de caducidad requiere lista negra o refresh tokens —no lo implementé en profundidad. Para el alcance académico, caducidad de 24h es razonable; en producción valoraría refresh o rotación.

3.5. Capa de presentación en Android frente a lógica

Compose anima a mantener la UI como función de estado. La lógica de negocio “de verdad” sigue estando en servidor para datos críticos; en cliente solo validaciones de formulario y reglas de UX. Si alguna regla existiera solo en el móvil, un usuario malintencionado podría saltársela llamando al API directamente —por eso **nunca** confío en el cliente para decidir si una marca es válida frente a baremo.

4. Estructura de carpetas del repositorio (mapa mental)

- **BBDD/** — `mydb.sql` crea el esquema; `seed.sql` (u otros scripts) cargan datos de prueba.
- **backend/** — `app.js` arranca Express; `src/routes/` define URLs; `src/controllers/` traduce HTTP; `src/services/` concentra lógica y SQL; `src/middleware/` para JWT; `src/config/` para pool de BD y Firebase Admin si aplica.
- **frontend/** — proyecto Android; bajo `app/src/main/java/com/opofit/miapp/` están `data` (API, modelos), `ui` (pantallas, navegación, viewmodels), etc.

Cuando busco un fallo, **primero** localizo la capa: ¿es la petición que no sale del móvil, la respuesta del servidor o el dato en tabla?

4.1. Más detalle del frontend Android (paquetes que más abrí)

Dentro de `com.opofit.miapp`, el `OpoFitApp` (o clase `Application` equivalente) suele inicializar cosas globales si las hay —por ejemplo dependencias o configuración de Firebase según el proyecto.

`MainActivity` es el contenedor de Compose: `setContent`, tema, y el `NavHost` principal o el punto donde se inyecta navegación.

`ui/navigation/AppNavigation.kt` es un archivo que toqué cada vez que añadí una pantalla nueva: cada `composable` con ruta y argumentos es un contrato. Si olvidé pasar un parámetro en el `navigate`, la pantalla recibe valores por defecto y el bug es silencioso —por eso prefiero rutas tipadas o constantes centralizadas en `NavDestinations.kt`.

`ui/screens/auth` — Login y registro: formularios, validación visual, llamadas a `AuthViewModel`. Aquí conviene que los errores del servidor se muestren tal cual (`msg`) para no desincronizar textos.

`ui/screens/main/MainScreen.kt` — Suele contener la barra inferior y los subdestinos (home, rutinas, perfil, historial según implementación). Es fácil que el estado de sesión viva aquí o en un `ViewModel` de ámbito superior.

`ui/screens/rutinas` — Crear, listar, detalles, rutinas libres: los archivos con más uniones con `RutinasViewModel` y endpoints `/api/rutinas*` y `/api/rutinas-pers*`.

`ui/screens/entrenamientos` — Flujo de entreno y registro final; coordina temporización y recolección de resultados antes del POST a historial.

`ui/screens/historial` — Listados y quizá filtros; conecta con `/api/historial/sesiones` y evolución.

`ui/screens/perfil` y `ajustes` — Perfil editable, settings; PUT a `/api/user/*`.

`data/api` — Interfaces Retrofit; si cambio base URL, lo hago en el módulo de red donde instancio `OkHttp/Retrofit` (buscar `Retrofit.Builder` o módulo DI si existe).

`utils` — Funciones auxiliares que no encajan en otra capa; revisar que no se conviertan en “basurero” de lógica de negocio que debería estar en servidor.

4.2. Backend: archivos que abro casi siempre al depurar

`app.js` — Orden de middlewares y rutas.

`src/routes/*.js` — Qué método HTTP y qué función.

`src/controllers/*.js` — Validación superficial y códigos HTTP.

`src/services/*.js` — SQL y reglas.

`src/middleware/authMiddleware.js` — Token.

`src/config/db.js` — Pool.

4.3. Convención de ramas Git (lo que intenté)

Feature branches por bloque (`feature/historial` , `fix/token` , etc.) y merge a principal cuando la feature compila y probé el flujo. No siempre fui disciplinado; cuando no, los commits largos me dolieron al hacer `bisect` .

5. Base de datos: visión por dominios

5.1. Oposiciones y pruebas

`oposiciones` — catálogo de oposiciones.

`pruebas_oficiales` — cada prueba (nombre, descripción, trucos, vínculo a oposición). El flag `mejor_si_es_menor` lo uso en la lógica de baremos: según el tipo de prueba, una marca “mejor” puede ser un tiempo más bajo o un valor más alto.

5.2. Usuario y ajustes

`usuarios` — datos de cuenta y perfil físico. El email es único. La contraseña va **hasheada** (bcrypt en el servidor), nunca en claro. El IMC lo calculo en registro a partir de peso y altura (altura en centímetros convertida a metros para la fórmula habitual).

`settings` — preferencias como unidades; se crea fila al registrarse con valores por defecto razonables (por ejemplo kg y km en mi implementación).

5.3. Marcas y baremos

`marcas_perfil` — mejor marca del usuario por prueba oficial; índice único por par (usuario, prueba) para no duplicar.

`requisitos_nivel` — umbrales por género y nivel de exigencia ligados a prueba.

`baremos_puntuacion` — tablas marca → nota por género y prueba. El servicio de rutinas cruza marcas del usuario con esta tabla para razonar notas medias y niveles sugeridos cuando los datos están completos.

5.4. Ejercicios y rutinas oficiales

`ejercicios` — catálogo compartido (nombre, vídeo, instrucciones).

`rutinas_opo` — cabecera: nivel (BÁSICO / INTERMEDIO / AVANZADO), género, `enfoque_tipo` (FUERZA / VELOCIDAD / RESISTENCIA), oposición.

`detalle_rutina_opo` — líneas: qué ejercicio, series, repeticiones, descanso.

5.5. Rutinas personalizadas

`rutinas_pers` — cabecera por usuario (nombre personalizado).

`detalle_rutina_pers` — líneas con ejercicios del catálogo y parámetros.

5.6. Historial y resultados

`historial_sesiones` — una fila por sesión completada: fecha, tipo (OPO o PERS), duración, referencia a rutina oficial o personal, usuario.

`registro_resultados` — valores numéricos por ejercicio en esa sesión. La evolución temporal se obtiene uniendo resultados con sesiones por fecha.

5.7. Noticias

`noticias` — puede alimentar contenido estático por oposición; en paralelo el servidor puede **agregar RSS** externos según configuración en `RssService`.

5.8. Relaciones que más me costaron explicar en voz alta

Entre `oposiciones` y `usuarios` hay una relación opcional: un usuario puede tener oposición asignada o no, según el momento del registro o ediciones posteriores. Esa flexibilidad tiene coste: en pantallas que asumen oposición siempre presente hay que manejar el caso null.

Entre `pruebas_oficiales` y `baremos_puntuacion` hay cardinalidad uno-a-muchos: una prueba tiene muchas filas de baremo (distintas marcas y notas). En consultas de info de pruebas hago `JOIN` explícito y ordeno para que la UI pueda agrupar mentalmente por nombre de prueba.

Entre `rutinas_opo` y `detalle_rutina_opo` la relación es uno-a-muchos: una cabecera de rutina, muchas líneas. Si borrara una cabecera sin borrar detalles, violaría integridad —por eso en un sistema real habría borrado en cascada o restricciones `ON DELETE CASCADE`; en mi esquema revisé las FK para saber qué permite MySQL tal como lo generé.

Las **rutinas personalizadas** repiten el patrón cabecera-detalle pero con `usuarios` como dueño. El historial referencia **o** rutina opo **o** rutina pers según `tipo_rutina`, dejando la otra FK a null. Ese diseño evita duplicar tablas de historial; la complejidad pasa a la aplicación: al insertar debo rellenar la columna correcta.

5.9. Tipos ENUM que usé y por qué no los dejé como texto libre

Nivel de rutina (`BASICO` , `INTERMEDIO` , `AVANZADO`), género (`HOMBRE` , `MUJER`), enfoque (`FUERZA` , `VELOCIDAD` , `RESISTENCIA`), tipo de rutina en historial (`OPO` , `PERS`) están en ENUM o equivalentes en el esquema. Preferí ENUM frente a texto suelto para que la base de datos **rechace basura** en inserciones accidentales. El inconveniente es que cambiar convenciones de nombres implica migración; dentro del TFG los valores son estables.

5.10. Campos DECIMAL y unidades

Peso, altura, marcas y valores de baremo usan tipos numéricos con decimales. Internamente el usuario piensa en kilos o segundos; la base guarda números. La coherencia de unidades es responsabilidad de la aplicación y de los textos mostrados —por eso `settings` y los textos de UI importan tanto como el tipo SQL.

5.11. Índices que me importan para consultas reales

Además de claves primarias y foráneas, hay índices únicos como el de email en usuarios y el único compuesto en marcas por usuario y prueba. Sin ese índice único, el upsert de marcas sería más frágil. Cuando depuré duplicados históricos, agradecí haber podido identificar filas por esas claves.

5.12. Qué haría distinto en una migración “de verdad”

Si el proyecto creciera, pasaría de un único `mydb.sql` a migraciones versionadas (Flyway, Liquibase o scripts numerados). Ahora mismo el esquema es “el snapshot” que generé y mantenía a mano cuando tocaba; reconozco que en equipo grande no bastaría.

5.13. Integridad frente a comodidad en desarrollo

A veces fue tentador relajar una FK para cargar datos rápido a mano; resistí en lo posible porque **los bugs más feos** que vi en proyectos académicos venían de datos inconsistentes que la app asumía imposibles. Prefiero error de inserción claro que pantalla que muestra “undefined” en producción.

5.14. Cómo leo el diagrama E-R cuando dudo

Si no sé dónde guardar algo nuevo, pregunté: ¿es un hecho del usuario? ¿un catálogo global? ¿un evento en el tiempo? Los hechos del usuario van hacia historial o marcas; el catálogo hacia ejercicios o pruebas; eventos temporales hacia sesiones y resultados. Si no encaja en ninguna categoría, o es un campo mal ubicado o necesito una tabla nueva.

5.15. Semillas de datos y orden de carga

Al cargar `seed.sql`, respeté el orden de tablas sin FK rotas: primero oposiciones, luego pruebas y baremos, luego ejercicios y rutinas que dependen de ellos. Si un compañero reproduce el proyecto y el script falla a mitad, el error suele ser **orden** o un id que ya existía —documento esto porque me pasó en algún momento durante pruebas.

6. Reglas de negocio que me importó dejar claras

6.1. Nivel sugerido y nota media (`RutinasService`)

El cálculo de **nota media** y **nivel sugerido** a partir de baremos **no debe engañar** si faltan marcas: si el número de pruebas completadas es menor que el total de pruebas de esa oposición, devuelvo información explícita de que faltan datos en lugar de forzar un nivel con información parcial. Eso lo implementé porque de lo contrario una sola marca buena podría disparar un nivel incoherente con la realidad del opositor.

6.2. Un entreno por rutina y día (`ProgresoService`)

Antes de insertar historial, compruebo si ya existe sesión **hoy** para el mismo usuario y la misma rutina (oficial o personal según el tipo). Si existe, lanzo error con mensaje entendible: el usuario no debería duplicar accidentalmente sesiones el mismo día para la misma rutina.

6.3. Unidades en ejercicios de cardio (`RutinasService.inferUnidad`)

Para mostrar bien duraciones o distancias en UI, inferí unidad a partir del nombre del ejercicio (minutos, segundos, km, metros, repeticiones por defecto). Es una heurística: no es perfecta, pero evita tratar una carrera como si fueran “repeticiones” sin más.

6.4. Zona horaria y “mismo día” en historial

La comprobación de duplicado diario en `ProgresoService` usa `DATE(fecha_entreno) = CURDATE()` en servidor MySQL. Eso ancla el “día” al **huso horario del servidor**, no necesariamente al del móvil. En un producto internacional movería la lógica a UTC o guardaría explícitamente la zona del usuario. En el TFG asumí entorno local único y servidor en mi máquina —si el tribunal pregunta, la respuesta honesta es ese límite.

6.5. Valores decimales y separadores

Los números van como DECIMAL/float en JSON. Cuidado con comas locales en UI: el cliente debería enviar punto o el servidor debería normalizar. Si alguna vez vi un 400 por formato numérico, fue por parseo en cliente.

6.6. Eliminación de rutina personalizada

Al borrar una rutina pers, debo asegurarme de que no queden filas en `detalle_rutina_pers` huérfanas ni historial inconsistente —según cómo estén las FK, el borrado puede fallar o requerir borrar en orden. Revisé `RutinaPersService` al implementar delete para no dejar basura.

6.7. Recuperación de contraseña y verificación de email

No son el núcleo del TFG; no las implementé como flujo completo con mailer. Si el tribunal pregunta, la respuesta es: **alcance y tiempo**; en producción serían obligatorias. Mientras tanto, el usuario puede usar login social si está habilitado.

6.8. Modo offline y sincronización

La app asume conectividad para casi todo: rutinas, historial, perfil. No implementé una base local Room/SQLite completa con sincronización bidireccional; eso sería otro proyecto entero (conflictos de edición, colas de envío, resolución de duplicados). Si el usuario pierde red a mitad de un formulario, lo razonable es mostrar error y no fingir que se guardó. En gimnasios con mala cobertura, la mejora natural sería cachear última rutina vista y permitir “borrador” local —anótalo como evolución, no como carencia accidental sin reflexión.

Mientras tanto, lo que sí tiene sentido en casi cualquier app es **no bloquear la UI** si la red va lenta: timeouts razonables en Retrofit y mensajes de reintento evitan que el usuario piense que el móvil se colgó. Yo mismo probé cortando datos móviles para ver que la app reaccionaba con error visible y no con pantalla en blanco eterna. Pequeños gestos de prueba así ahorran vergüenza en la defensa oral ante el tribunal.

7. Backend: `app.js` y middleware global

En `backend/app.js` configuro:

- `dotenv` para variables de entorno.

- **CORS** — en desarrollo abierto; en producción habría que restringir orígenes.
- `express.json()` para parsear cuerpos JSON.
- `express.static('public')` si hay assets estáticos.
- **Rate limiting global** — ventana de 15 minutos, máximo de peticiones por IP (mensaje en español).
- **Rate limiting extra en** `/api/auth` — máximo más bajo para rutas de login/registro y variantes, frente a fuerza bruta.

Después **monto las rutas** bajo prefijos: `/api/auth`, `/api/user`, `/api/oposiciones`, `/api/rutinas`, `/api/rutinas-pers`, `/api/historial`, `/api/info`, `/api/ejercicios`.

El **puerto** por defecto es 3000 o el que venga en `PORT`.

8. Autenticación: email y contraseña, JWT

8.1. Registro (`AuthService.registrar`)

Flujo resumido:

1. Normalizo email (`trim`, minúsculas).
2. Si viene oposición, verifico que exista en `oposiciones` —evito insertar usuarios con FK rota.
3. Hasheo password con `bcrypt` (coste 10 en mi código).
4. Calculo IMC y abro **transacción**.
5. Inserto usuario, inserto `settings` por defecto, opcionalmente marcas iniciales si el cliente las manda.
6. `commit` o `rollback` si algo falla.

El controlador, tras registro exitoso, emite **JWT** firmado con `JWT_SECRET` y caducidad (p. ej. 24h). La respuesta incluye token y datos de usuario **sin** el campo password.

8.2. Login

Busco usuario por email normalizado, comparo password con `bcrypt.compare`, elimino password del objeto devuelto y firmo JWT con id y email.

8.3. Google (OAuth)

Verifico el **ID token** de Google con `google-auth-library` y `GOOGLE_CLIENT_ID`. No me fío del email que llega “suelto”: el token tiene que coincidir. Si el usuario existe, lo devuelvo; si no, hay flujos de alta que crean usuario con password interna aleatoria hasheada (sigue existiendo columna password en BD).

8.4. Firebase

Con **Firebase Admin** verifico el `idToken`. El comportamiento exacto de login vs registro depende de los métodos `loginConFirebase` / `registrarConFirebase` en `AuthService`: la idea general es **no crear perfil completo sin datos** cuando la política del proyecto exige oposición y datos físicos en el alta.

8.5. Endpoint `/api/auth/me`

Con middleware `validarToken`, devuelvo información del usuario autenticado para refrescar UI (perfil, etc.).

9. Middleware `validarToken` (cómo interpreto el JWT)

En `backend/src/middleware/authMiddleware.js` leo la cabecera `Authorization`. Si no viene, 401 con mensaje claro en español. Si viene, espero formato `Bearer <token>` y tomo la segunda parte tras el espacio.

Verifico con `jwt.verify(token, JWT_SECRET)`. Si `JWT_SECRET` no está definido, respondo 500 —prefiero fallar ruidoso en configuración antes que firmar mal. Si verifica bien, guardo el payload en `req.usuario` y llamo a `next()`.

Detalle importante: en `RutinaController.getMiEntrenamiento` uso `req.usuario.id` como fuente de verdad del usuario autenticado, no el `userId` de la URL (de hecho el de params puede ignorarse). Esa fue una decisión consciente para **evitar** que alguien manipule la ruta y pida entrenamiento de otro id.

10. Mapa de rutas HTTP (qué toca cada prefijo)

10.1. `/api/auth`

Método y ruta	Qué hace en la práctica
POST <code>/registrar</code>	Alta con email/password y datos de perfil; devuelve JWT si todo va bien.
POST <code>/login</code>	Comprueba credenciales; devuelve usuario sin password y JWT.
POST <code>/google</code>	Login con token de Google verificado en servidor.
POST <code>/google/registrar</code>	Alta vía Google cuando no existía usuario.
POST <code>/google/firebase</code>	Login con ID token de Firebase verificado.
POST <code>/google/firebase/registrar</code>	Registro completo vía Firebase con perfil en cuerpo.
GET <code>/me</code>	Requiere <code>validarToken</code> ; datos del usuario actual.

10.2. `/api/user`

Rutas con `validarToken`: actualizar perfil, actualizar settings, eliminar cuenta. Aquí conviene que el backend compruebe que el recurso pertenece al `req.usuario.id`.

10.3. `/api/oposiciones`

- GET `/` — listado de oposiciones (puede ser público según implementación; revisa el controlador si cambias seguridad).
- GET `/rss/:id` — noticias agregadas por RSS para esa oposición (token).
- GET `/requisitos/:id/:genero` — requisitos (token).
- GET `/:id` — información detallada de oposición (token).

10.4. `/api/rutinas`

- GET `/mi-entrenamiento/:userId/:idOposicion` — **corazón** de la rutina oficial sugerida: calcula nivel, obtiene rutina completa o devuelve estado “incompleto” si faltan marcas.

10.5. /api/rutinas-pers

Crear rutina personalizada, listar por usuario, eliminar por usuario e id de rutina —siempre con token.

10.6. /api/historial

Registrar entreno, evolución por ejercicio, listado de sesiones por usuario.

10.7. /api/info

Información de pruebas y marcas del usuario frente a oposición.

10.8. /api/ejercicios

Listado del catálogo para construir rutinas libres.

11. RutinaController.getMiEntrenamiento paso a paso (mi lectura del código)

Este endpoint es de los más densos porque mezcla **cálculo de nivel**, **mensajes de negocio** y **consulta de rutina completa**:

1. Obtengo `userId` del **token** (`req.usuario.id`), no de la URL.
2. Llamo a `RutinaService.calcularNotaYNivel(userId, idOposicion)`.
3. Si el servicio dice usuario no encontrado, devuelvo 401 coherente con sesión inválida.
4. Si hay **pruebas faltantes**, no es error 500: respondo 200 con `nivelAsignado` tipo INCOMPLETO, mensaje amable y datos vacíos de rutina —así la app puede enseñar “completa tu perfil” sin romper el flujo.
5. Si hay nivel y género, pido `obtenerRutinaCompleta(idOposicion, nivelSugerido, genero)` y devuelvo la estructura que la app pinta en detalle de rutina.

Este patrón de “200 con estado incompleto” frente a “400 por todo” es una decisión de **UX**: el usuario entiende qué le falta en lugar de ver solo error genérico.

11.1. Qué hace exactamente `obtenerRutinaCompleta` (lectura recomendada del servicio)

En `RutinasService.obtenerRutinaCompleta` primero busco filas en `rutinas_opo` que coincidan en **oposición**, **nivel** y **género**. Uso `SELECT DISTINCT id_rutina_opo, enfoque_tipo` porque para una misma terna pueden existir **varias rutinas** que difieren en el **enfoque** (fuerza, velocidad, resistencia). Eso encaja con la idea de que el usuario elija qué priorizar en un momento dado: no es lo mismo una sesión con sesgo a fuerza que una con sesgo a resistencia.

Para cada fila `r` devuelta por esa consulta, abro un segundo query que une `detalle_rutina_opo` con `ejercicios` filtrando por `rutinas_opo_id_rutina_opo`. Así obtengo, por cada bloque de enfoque, la lista de ejercicios con series, repeticiones y descanso, más metadatos del ejercicio (nombre, vídeo).

Construyo un array `planCompleto` donde cada elemento tiene `id_rutina_opo`, `bloque` (el `enfoque_tipo`) y `ejercicios`, y para cada ejercicio añado `unidad inferida` con `inferUnidad(nombre)`. Esa inferencia no es perfecta —depende de cómo estén nombrados los ejercicios en base de datos— pero evita que la UI trate una carrera como repeticiones sin contexto.

Si la primera consulta no devuelve filas, retorno `null` y el controlador traduce eso en 404 con mensaje de rutina no encontrada para ese nivel: señal de que **falta contenido** cargado en `rutinas_opo / detalle_rutina_opo` para esa combinación, no necesariamente de que el usuario esté “mal”.

11.2. Relación entre enfoque del usuario y filas devueltas

En la navegación de la app pasé el parámetro de enfoque hacia entrenamientos según la UI (`EntrenamientosScreen / callbacks` desde `MainScreen`). En el backend, la selección final de qué bloque mostrar puede depender de cómo filtré las rutinas en servicio o controlador —lo importante es que **conceptualmente** el usuario no reciba una mezcla incoherente: o ve un bloque alineado con su elección, o un mensaje que explique por qué no hay nada. Si en algún momento mezclé lógica en cliente y servidor, el sitio de verdad es el resultado JSON que recibe la pantalla de detalle.

11.3. Por qué el controlador ignora `userId` de la URL

Es una decisión de seguridad: el identificador real lo lleva el token. El parámetro `:userId` puede existir por compatibilidad de rutas o por historial de refactor; lo que importa es que **no deba poder sustituirse** por otro número para espiar datos. Cuando revisé esto, confirmé que el cálculo y la rutina usan `req.usuario.id`. Si encuentras código viejo que aún confía en el path, es candidato a refactor.

12. `ProgresoService.registrarEntreno` — transacción y regla diaria

Entrada típica: `userId`, `tipoRutina` (OPO/PERS), `idRutina`, `duracion`, lista de `{ id_ejercicio, valor }`.

1. Compruebo duplicado **mismo día** misma rutina y usuario (consulta dinámica según tipo).
2. `beginTransaction`.
3. Inserto en `historial_sesiones` con la FK correcta (columna rutina opo o pers según tipo).
4. Con el `insertId`, inserto cada resultado en `registro_resultados`.
5. `commit` o `rollback`.

Si algo falla después de empezar transacción, hago `rollback` y propago error para que el controlador responda 500 o el mensaje que toque.

13. RSS (`RssService`) — expectativas realistas

Configuré mapas de feeds por id de oposición (BOE, Guardia Civil, etc.). El parser tiene timeout y user-agent razonable. Límite items por feed y total para no devolver megabytes de HTML incrustado. Recorto descripciones largas para la tarjeta de noticia.

En la vida real los feeds cambian, devuelven HTML raro o van lentos. Por eso el código defensivo y, si hace falta, logs con `RSS_DEBUG`. Yo asumí que **a veces** no habrá noticias y eso no puede tumbar la app.

14. Tests que recuerde haber tocado

Hay pruebas en `backend/tests/` para el middleware de token (casos con header ausente, mal formado, token inválido). No es cobertura total, pero me dio confianza al tocar auth.

14 bis. Casos de uso largos (cómo los expliqué yo mismo en voz alta)

Caso 1 — Registro desde cero. El usuario elige email y contraseña, rellena nombre, género, peso, altura y oposición. El servidor valida oposición, calcula IMC, inserta usuario y settings, y opcionalmente marcas iniciales si el formulario las envía. Recibo JWT y navego a la pantalla principal. Si el email ya existía, el servidor responde conflicto y muestro mensaje sin crear usuario fantasma.

Caso 2 — Login tras cerrar la app. Al abrir, si el token sigue vigente, entro directo al main; si caducó, vuelvo a login. Dependiendo de cómo guardé el token, puede haber estado intermedio “sesión inválida” que resuelvo forzando login limpio.

Caso 3 — Consultar rutina oficial. Llamo a mi entrenamiento con oposición actual. Si faltan marcas de alguna prueba, recibo estado incompleto: la UI debe explicar qué falta sin parecer un error de servidor. Si está completo, recibo nivel sugerido y lista de ejercicios con parámetros para entrenar.

Caso 4 — Crear rutina libre. Cargo catálogo de ejercicios, el usuario compone líneas, envío nombre y lista al servidor. El servidor valida que el userId del cuerpo coincide con el token, inserta cabecera y detalles, devuelve id. Vuelvo a la lista y veo la rutina nueva agrupada.

Caso 5 — Entrenar y registrar. Tras la sesión, envío tipo de rutina, id de rutina, duración y resultados por ejercicio. Si intento registrar dos veces el mismo día la misma rutina, el servidor rechaza con mensaje claro. Si va bien, veo la sesión en historial.

Caso 6 — Ver evolución. Elijo un ejercicio; el servidor devuelve puntos (fecha, valor) ordenados. La UI puede mostrar lista o gráfico simple según lo implementado. Lo importante es que los datos vienen de `registro_resultados` unidos a sesiones.

Caso 7 — Editar perfil con nuevas marcas. Cambio peso o añado marcas; el servidor hace upsert y limpieza defensiva si hiciera falta. Si mando oposición, recibo cálculo de nivel actualizado para refrescar sensación de progreso.

Caso 8 — RSS. Abro info de oposición; si hay noticias, las veo; si no, no es catástrofe: sigo teniendo descripciones de pruebas desde endpoints estáticos.

14 ter. Detalle de cada grupo de rutas (más lento, más explícito)

Auth: aquí vive todo lo que **no** requiere sesión previa salvo `/me`. Cuando añadí Google y Firebase, dupliqué mentalmente el flujo: primero verificar token externo, luego decidir si creo usuario o devuelvo error de “no registrado”. El JWT propio lo genero **después** de esa verificación, igual que en login clásico.

User: todo lo que muta perfil o borra cuenta debe estar ligado al token. Si algún día permitiera roles admin, separaría prefijos; hoy el modelo es un único tipo de usuario.

Oposiciones: el listado público o protegido depende de la decisión en controlador; yo lo documento así porque en defensa me pueden preguntar “¿por qué cualquiera puede listar oposiciones?”. Mi respuesta razonada sería: es catálogo no sensible; lo sensible son datos de usuario.

Rutinas oficiales: un solo endpoint clave con mucha lógica en servicio; merece la pena leer `RutinaController` entero al menos una vez en la vida del proyecto.

Rutinas pers: tres operaciones CRUD-like; la eliminación debe borrar detalles o impedir huérfanos según implementación del servicio —revisé `RutinaPersService` cuando depuré borrados.

Historial: escritura transaccional y lecturas de listado/evolución. Separar lecturas evita bloqueos innecesarios en escritura si el motor lo requiere; con el volumen del TFG no fue problema.

Info pruebas: joins pesados en lectura; si un día optimizo, sería con vistas materializadas o caché; no urgente ahora.

Ejercicios: lectura simple; base para todo lo demás de rutinas libres.

14 quater. Cómo encajan Retrofit y los interceptores (si los usé)

Retrofit genera implementación de mis interfaces. Si añadí interceptor para inyectar `Authorization`, centralizo el header. Si no, cada llamada del ViewModel añade el header manualmente —menos elegante pero más explícito para aprender. Revisa el código real para saber cuál hice; ambas son válidas en TFG.

14 quinquies. Compose: recuerdos de estado compartido

Cuando varias pantallas necesitan el mismo usuario, puede vivir en un ViewModel de ámbito superior o pasarse por navegación. Elegí un enfoque según complejidad; si mezclé demasiado estado global, la depuración se complicó. Lección: **menos estado mágico**, más props explícitas cuando el flujo es lineal.

15. Android: paquetes y responsabilidad

15.1. `data/api`

Interfaces Retrofit (`AuthApi`, etc.) con rutas que coinciden con el backend. Uso `suspend` para integrarme con corrutinas.

15.2. `data/models` y `data/responsemodels`

Separo **cuerpos de petición** y **formas de respuesta**. Cuando el backend añade un campo opcional, actualizo el modelo Kotlin y listo —el compilador me avisa donde falta mapear.

15.3. `data/local`

`TokenManager`, `SessionManager` (o equivalentes) — persistencia del JWT y estado de sesión. Aquí decidí cómo guardar el token de forma razonable para un TFG (mejor que en `SharedPreferences` plano si usé cifrado; revisa el código concreto).

15.4. `ui/navigation`

`NavDestinations` con rutas string; `AppNavigation` monta el `NavHost`, decide destino inicial login vs main según sesión, y define callbacks de navegación desde `MainScreen`.

15.5. `ui/screens` y `ui/viewmodels`

Cada flujo gordo tiene pantalla + ViewModel. La pantalla observa estado; el ViewModel llama APIs y maneja errores.

15.6. Tema, tipografía y consistencia visual

Intenté que colores y tipografías fueran coherentes entre pantallas (Material 3 / tema por defecto de Compose según configuración del proyecto). No soy diseñador gráfico; prioricé legibilidad y contraste razonable. Si un texto quedaba demasiado pequeño en dispositivos pequeños, lo detecté probando en emulador con distintas densidades.

15.7. Iconografía y recursos

Los iconos de navegación y botones siguen el set Material cuando fue posible. Evité iconos ambiguos para acciones destructivas (borrar rutina, borrar cuenta): confirmación con diálogo.

15.8. Gestión de teclado y formularios

En formularios largos, el teclado puede tapar campos; en Compose se puede usar `imePadding` o scroll según versión. Si algo quedó cortado en mi dispositivo de prueba, lo anoté como mejora.

15.9. Accesibilidad (nivel básico)

Intenté tamaños tocables razonables y textos no solo por color. Una auditoría WCAG completa no está hecha; lo reconozco para no vender humo en la defensa.

15.10. Pruebas manuales en dispositivo real vs emulador

El emulador es rápido para iterar; el dispositivo real descubre problemas de red, rendimiento y sensación táctil. Alterné ambos cuando pude.

15.11. Empaquetado release vs debug

El build release firma y optimiza; requiere keystore y cuidado con `minify` si activo R8/ProGuard —las reglas de Retrofit pueden necesitar keep. En TFG a veces me quedé en debug para iterar más rápido.

15.12. Versiones mínimas de SDK

Lo fijé en `build.gradle.kts` según librerías usadas; si subo `minSdk`, corto dispositivos viejos; si bajo, puede que no compile.

16. Pantallas que yo mismo enlace mentalmente con endpoints

- **Login / Registro** ↔ `/api/auth/*`
- **Main / Home** ↔ navegación y atajos
- **Info oposición** ↔ `/api/oposiciones/:id`, RSS, requisitos
- **Rutinas / Detalle rutina oficial** ↔ `/api/rutinas/mi-entrenamiento/...`
- **Rutinas libres / Crear** ↔ `/api/ejercicios`, `/api/rutinas-pers/*`
- **Entrenamiento / Registrar** ↔ `/api/historial/registrar`
- **Historial** ↔ `/api/historial/sesiones/...`, evolución
- **Perfil / Editar** ↔ `/api/user/perfil`, `/api/auth/me`
- **Ajustes** ↔ `/api/user/settings`

No hace falta que memorice números de línea: hace falta que cuando falle algo sepa **qué fichero** abrir.

17. Flujo completo: desde login hasta guardar entreno (relato operativo)

1. **Login.** El usuario introduce credenciales. El ViewModel deshabilita el botón o muestra carga para evitar doble envío. Si el servidor responde bien, guardo token y usuario mínimo; si mal, muestro el mensaje (`msg`) que viene del backend cuando existe.
 2. **Arranque de la zona autenticada.** Al entrar en `MAIN` , puede dispararse una llamada a `/api/auth/me` para alinear UI con datos frescos (peso, oposición, etc.). Si falla por token, redirijo a login.
 3. **Decisión de entrenamiento.** Desde home o accesos directos, el usuario elige enfoque o flujo. La navegación pasa parámetros en la ruta de Compose; es fácil equivocarse con strings —por eso centralicé rutas en `NavDestinations` .
 4. **Petición de rutina oficial.** La pantalla de entrenamientos solicita la rutina con oposición del usuario. Hasta que no llega respuesta, muestro skeleton/spinner. Si el estado es incompleto por marcas faltantes, cambio el copy: no es “error”, es **acción sugerida** (ir a perfil).
 5. **Ejecución física (fuera del software).** El móvil no cuenta repeticiones por sensor en mi alcance típico: el usuario introduce valores o tiempos según diseño de UI. Ahí la UX importa: teclado numérico, validación de vacíos, mensajes si el valor es absurdo.
 6. **Montaje del payload de historial.** Debo incluir tipo de rutina (OPO/PERS), id de rutina coherente con el tipo, duración en el formato que espera el backend, y lista de `{ id_ejercicio, valor }` . Si mezclo ids de otra rutina, los resultados serán incoherentes pero quizá no fallen FK —por eso la disciplina al construir la lista desde la misma sesión visualizada.
 7. **POST historial.** Si recibo error de duplicado diario, muestro el mensaje literal del servidor; es una regla de negocio, no un fallo de red.
 8. **Confirmación en historial.** Tras éxito, refresco lista local o navego a historial; depende del patrón que implementé. Lo importante es que el usuario **vea** el efecto de su acción; si no, pensará que no guardó.
 9. **Evolución (opcional).** Si el usuario abre evolución de un ejercicio, otra llamada une datos históricos. Puede ser lenta si hay muchas sesiones —en TFG no optimicé paginación.
- Caducidad de token:** si entre pasos pasaron horas y el JWT expiró, los pasos 3–9 pueden devolver 401 en cascada. Mi manejo esperado: logout limpio y mensaje “sesión expirada”.
-

18. Variables de entorno que no quiero olvidar

Ejemplos típicos en `.env` del backend:

- Credenciales **MySQL** (host, usuario, password, base).
- `JWT_SECRET` — debe ser largo y secreto; sin esto no arranca auth bien.
- `GOOGLE_CLIENT_ID` — para verificar tokens de Google.
- Configuración **Firebase Admin** (ruta a JSON de credenciales o variables que use `firebaseAdmin.js`).
- Opcional `RSS_DEBUG` para depurar feeds.

Si algo “funciona en mi PC” y no en otra, el 80% de las veces es `.env` incompleto o IP mal puesta en el cliente Android.

18.1. Cómo documenté el `.env` para no filtrar secretos

Mantuve un `.env.example` en el repo con claves vacías o valores placeholder y el `.env` real fuera de git. Nunca subí `JWT_SECRET` real ni credenciales de Firebase. Si alguna vez pegué un token en un chat o commit, lo consideraría comprometido y lo rotaría —paranoia sana.

18.2. Puerto y despliegue

`PORT` controla el puerto HTTP. En desarrollo 3000 es cómodo; en producción detrás de nginx podría ser interno distinto. El cliente Android debe apuntar al host/puerto correctos; un error típico es olvidar actualizar la URL al cambiar de Wi-Fi o de VPN.

18.3. Variables que el frontend “no ve”

El móvil no necesita `JWT_SECRET`; solo necesita la URL base y quizá claves públicas de Firebase/Google según SDK. **Nunca** empaquetes secretos del servidor en la APK.

18.4. Rotación de secretos

Si rotara `JWT_SECRET`, todos los tokens emitidos antes invalidan instantáneamente —los usuarios cerrarían sesión. En producción planificaría rotación con ventana de doble secreto o refresh tokens; aquí no aplica.

19. Cómo levanto el entorno local (mi checklist)

1. **MySQL** encendido y servicio estable. Creo la base si no existe, ejecuto `mydb.sql` y luego semillas. Si algo falla a mitad de script, reviso el mensaje de FK: casi siempre es orden de tablas o datos que referencian ids inexistentes.
2. **Usuario de base de datos** con permisos suficientes (`SELECT`, `INSERT`, `UPDATE`, `DELETE` al menos sobre el schema del proyecto). En desarrollo a veces uso `root` local; en producción usuario dedicado con mínimo privilegio.
3. **Backend:** `npm install` en la carpeta correcta (no mezclar con `Documentacion/node_modules` si generé PDFs ahí). Copio `.env.example` → `.env` y relleno credenciales. Verifico que `JWT_SECRET` exista antes de probar login.
4. **Arranque:** `node app.js` o script npm. Leo la consola: si dice puerto en uso, mato el proceso viejo o cambio `PORT`.
5. **Prueba rápida:** `curl` o Postman a una ruta pública/listado si existe, o health check si lo añadí. Si ni eso responde, no abro Android todavía.
6. **Firewall de Windows:** si el móvil físico no llega, permitir Node o el puerto en firewall temporalmente para desarrollo.
7. **Android:** URL base con IP LAN de la PC (comando `ipconfig`). Desde dispositivo físico `localhost` apunta al propio teléfono, no al PC.
8. **Emulador:** `10.0.2.2` mapea al host; si uso ese truco, verifico que el servidor escuche en `0.0.0.0` o al menos en la interfaz accesible (a veces solo `localhost` en servidor impide conexión externa — depende de cómo arranque Express).

9. **HTTPS:** en local no lo usé; si el día de mañana fuerzo HTTPS en servidor, el cliente Android necesitaría configuración de red segura o certificados de confianza.
 10. **Limpieza:** si cambio schema, a veces conviene dropear y recrear base en desarrollo antes de perseguir bugs fantasma por migraciones a medias.
-

20. Problemas que sufrí y cómo los encajé

- **CORS / red:** si el móvil no llega, ping y firewall.
- **Token caducado:** vuelta a login; no es bug, es expiración.
- **RSS vacío:** fuente caída o parseo raro; no debe bloquear rutinas.
- **Nivel incompleto:** no es fallo del cálculo; es falta de datos en `marcas_perfil`.
- **Gradle / Firebase:** si cambio `google-services.json`, limpio y rebuild.

Más detalle, porque a mí me hubiera ayudado leer esto antes:

- **“Connection refused” desde el móvil:** casi nunca es Compose; es IP, puerto o servidor apagado. Verifiqué que Express estuviera escuchando y que no hubiera otro proceso robando el puerto.
 - **401 en cadena tras un cambio de `JWT_SECRET`:** todos los tokens antiguos invalidados; cierro sesión en app y vuelvo a login.
 - **Datos duplicados en marcas:** apareció cuando intenté `INSERT` repetido sin upsert; la solución fue la lógica de update-then-insert más limpieza defensiva en `actualizarPerfil`.
 - **Rutina vacía pese a tener nivel:** faltaban filas en `detalle_rutina_opo` para esa cabecera — contenido, no lógica.
 - **Error SQL “Unknown column”:** desincronización entre código y base antigua; reaplicar migración o recrear schema.
 - **Timeouts en RSS:** subí timeout del parser y reduje expectativas: mejor lista corta que petición colgada.
-

20 bis. Perfil de usuario: lo que hice en `UsuarioController.actualizarPerfil`

Este endpoint es un buen ejemplo de **lógica real** que no sale en los tutoriales mínimos. Recibo `peso`, `altura`, `oposicionId` opcional y `nuevasMarcas` como lista. Primero valido que no mandes una petición vacía: tiene que haber al menos peso/altura válidos o marcas nuevas.

Si actualizo peso y altura juntos, recalculo IMC con la misma fórmula que en el registro. Si solo uno de los dos cambia, leo el otro valor desde base de datos para no dejar IMC incoherente o para no pisar con ceros por campos vacíos en el cliente.

Las **marcas** las trato con un patrón upsert: intento `UPDATE` por usuario y prueba; si no afecta filas, hago `INSERT`. Incluso añadí **limpieza defensiva** por si en el pasado hubiera duplicados históricos: conservo el registro más reciente y borro el resto para esa combinación usuario–prueba. Eso lo documento porque explica por qué hay un `DELETE` aparentemente agresivo: **nació de un problema real** de datos duplicados, no de capricho.

Tras actualizar, si me pasan `oposicionId`, vuelvo a llamar a `calcularNotaYNivel` para devolver al cliente un resumen actualizado del nivel —así la pantalla de perfil puede refrescar sensación de progreso sin otra ronda manual de peticiones.

20 ter. Rutinas personalizadas: permisos y agrupación

En `RutinaPersController` al crear rutina comparo `parseInt(userId)` del cuerpo con `req.usuario.id`. Si no coinciden, **403**. Lo mismo al listar rutinas por `:userId`. No confío en el id de la URL como “dueño” del token: el token manda.

Al listar, el servicio devuelve filas planas; el controlador agrupa en un `Map` por `id_rutina_pers` y construye objetos con array de ejercicios. Esa transformación en el controlador mantiene el SQL del servicio más simple y la respuesta JSON lista para Compose.

20 quater. Información de pruebas y marcas (`InfoPruebasService`)

`getInfoPruebas` une `pruebas_oficiales` con `baremos_puntuacion` filtrando por género y oposición, ordenando para que la UI pueda pintar tablas legibles.

`getMarcasUsuario` cruza marcas del usuario con pruebas de la oposición y añade un campo derivado `unidad` según `mejor_si_es_menor` (aproximación para tiempos vs repeticiones). Es una simplificación a propósito: la UI puede mostrar algo coherente sin que el usuario memorice reglas.

21. Más detalle del cálculo de nota y nivel (`RutinasService.calcularNotaYNivel`)

Para cada marca del usuario en pruebas de esa oposición, busco en `baremos_puntuacion` la mejor nota que corresponda según `mejor_si_es_menor`: si es mejor un valor menor (tiempos), la consulta exige que la marca sea menor o igual que el umbral del baremo; si es mejor un valor mayor, al revés. Me quedo con la nota más alta compatible (orden `DESC LIMIT 1`).

La media de notas alimenta el nivel: por debajo de cierto umbral básico, entre dos umbrales intermedio, y avanzado arriba. Si no hay notas para alguna prueba, el contador refleja que faltan datos.

Si quieres **reimplementar** esto, mi consejo es: primero escribe pruebas unitarias con datos de baremo mínimos inventados; después conectas a MySQL.

22. Consultas útiles cuando depuro (las uso en Workbench o similar)

Para ver sesiones recientes de un usuario:

```
SELECT id_historial_sesion, fecha_entreno, tipo_rutina, duracion_oficial
FROM historial_sesiones
WHERE usuarios_id_usuario = ?
ORDER BY fecha_entreno DESC
LIMIT 20;
```

Para ver resultados de una sesión:

```
SELECT e.nombre, r.valor_conseguido
FROM registro_resultados r
JOIN ejercicios e ON r.ejercicios_id_ejercicio = e.id_ejercicio
WHERE r.historial_sesiones_id_historial_sesiones = ?;
```

La evolución por ejercicio ya está encapsulada en `ProgresoService.obtenerEvolucionEntreno`, pero ver el SQL equivalente ayuda a entender los joins.

23. Android: `AppNavigation` y sesión

El `NavHost` arranca en LOGIN o MAIN según `isLoggedIn`. Al hacer login exitoso hago `popUpTo(LOGIN)` { `inclusive = true` } para que el botón atrás no vuelva al formulario con datos sensibles en pantalla. Al hacer logout limpio sesión en el `ViewModel` y reseteo la pila de navegación.

Las rutas con argumentos (`detalles_rutina/{rutina_id}`) las parseo a entero con cuidado: si viene basura, decido un fallback o pantalla de error según la pantalla.

24. ViewModels: patrón mental que seguí

No mezclé corrutinas en la UI pura: el `ViewModel` expone estado (`Loading / Success / Error`) o `StateFlow`. Cuando llamo a la API, envuelvo en `try/catch` y mapeo excepciones de Retrofit a mensajes que pueda mostrar en un `Snackbar` o diálogo.

Si el token expira, muchas llamadas devolverán 401: en algún sitio centralicé el “logout forzado” o lo manejé por pantalla —revisa la implementación concreta de interceptores si los añadí; si no, cada `ViewModel` trata el error. Lo ideal a futuro sería un interceptor `OkHttp` que detecte 401 una vez.

25. Modelos Kotlin y cambios en el backend

Cuando cambié un campo en la respuesta JSON, actualicé la data class en `responsemodels` y recompilé: Kotlin me obligó a revisar los usos. Eso es más seguro que un `Map` suelto donde el error sale en runtime.

26. Firebase y Google en el móvil (visión de integración)

En el cliente, el flujo típico es: obtengo credencial de Google/Firebase según el SDK, extraigo id token, lo envío al backend en el cuerpo que espera `AuthApi`. El servidor valida y devuelve el mismo estilo de respuesta que login clásico (usuario + JWT de mi API). **Mi JWT sigue siendo el de la aplicación**, no reemplazo toda la seguridad por el token de Google en cada petición sucesiva: uso el JWT propio como en el flujo email/password.

27. Seguridad: lista mental que repaso antes de entregar

- ¿Algún endpoint devuelve datos sin token que deberían ser privados?
- ¿Algún `userId` en path se confía sin comparar con token? (`RutinaPers` y `RutinaController` ya muestran el patrón correcto.)

- ¿`.env` está fuera del control de versiones?
 - ¿Las contraseñas nunca se loguean?
-

28. Datos semilla (`seed.sql`) y por qué importan

Sin datos de oposiciones, pruebas y baremos, la app “funciona” pero está vacía. Yo me aseguré de tener un juego mínimo creíble para demos. Si el tribunal pregunta por convocatorias reales, aclaro que los números son **ejemplo** cargados para desarrollo salvo que hayamos documentado fuentes oficial.

29. Errores HTTP que devuelvo y qué significan para el cliente

- **400** — validación: falta nombre de rutina, lista vacía, etc.
 - **401** — token ausente o inválido; sesión.
 - **403** — autenticado pero intentando recurso de otro usuario.
 - **404** — recurso no encontrado (rutina inexistente para ese nivel).
 - **409** — conflicto típico en registro (email duplicado).
 - **500** — algo inesperado en servidor; miro consola.
-

30. Rendimiento y límites (expectativas realistas)

Con los volúmenes de un TFG, no optimicé índices más allá de los que ya da el esquema. Si el historial creciera años, habría que paginar listados y añadir índices por `fecha_entreno`. Lo dejo escrito para el futuro yo mismo.

31. Qué cambiaría si empezara otra vez (sin dramatismo)

Unificar validación de cuerpos JSON con esquema declarativo. Extraer más tests de integración sobre historial. Documentar endpoints en OpenAPI cuando el tiempo lo permita. Nada de esto invalida lo entregado; es **deuda** consciente.

32. Pantallas principales y qué hacen (mi recorrido por `ui/screens`)

Esta sección es la que más me gusta releer porque conecta **carpetas** con **sensación de uso**. No es una lista de marketing: es lo que yo esperaba que un usuario tocara en un día normal de uso.

Login y registro — Recogen credenciales o datos de alta; llaman a `AuthViewModel`. Aquí cuidé mensajes claros cuando el servidor devuelve email duplicado u oposición inválida. El registro con datos físicos largos lo dividí mentalmente en “quién eres” y “qué oposición preparas” aunque visualmente sea un solo flujo. Si el teclado tapaba campos, hice scroll o ajusté padding según lo que viera en el emulador.

Main / Home — Punto de entrada tras login: atajos a entrenamientos, rutinas libres, información de oposición, perfil. La barra inferior organiza las secciones que consideré “diarias” frente a las secundarias. Desde aquí también puede salir el cierre de sesión; limpiar token y navegar a login debe

ser imposible de “deshacer” hacia atrás con el botón físico del teléfono si ya hice `popUpTo` correctamente.

Entrenamientos — Conecta con la lógica de enfoque (fuerza, velocidad, resistencia) según cómo lo pasé por navegación (`enfoque` en query). Es el puente entre intención del usuario y la petición de rutina. Si el parámetro llega vacío, decidí un comportamiento por defecto razonable o un mensaje —lo revisé para no mandar strings basura al backend.

Registrar entrenamiento — Aquí el usuario confirma duración y resultados por ejercicio. Es crítico que los ids de ejercicio coincidan con los que vienen de la rutina mostrada; si mezclas rutinas sin refrescar estado, puedes mandar resultados incoherentes —lo probé a mano. También validé que la duración no sea cero si el backend lo rechaza, o que al menos el usuario vea por qué no puede enviar.

Historial — Lista sesiones ordenadas por fecha. Si la UI muestra tipo OPO/PERS, ayuda a entender de dónde salió cada sesión. En sesiones vacías muestro estado amable (“aún no hay entrenos”) en lugar de pantalla en blanco intimidante.

Perfil y editar perfil — No solo cosmética: al actualizar marcas o peso/altura disparo el recálculo de nivel en el backend cuando mando `oposicionId`. Esa coordinación es fácil de romper si el cliente olvida un campo. Tras guardar, me gusta refrescar datos del usuario en memoria para que la UI no muestre valores viejos.

Ajustes — Persistencia de unidades; debe alinearse con lo que espera el servidor en `settings`. Si cambio kg a lb en el futuro, habría que definir si convierto datos históricos o solo etiquetas —fuera del alcance actual.

Oposición / info — Mezcla datos locales y RSS. Si falla la red de noticias, no debería bloquear la lectura de descripciones de pruebas ya cargadas desde API. Mostrar un pequeño aviso “noticias no disponibles” es mejor que spinner infinito.

Rutinas libres y crear rutina — Flujo de dos tiempos: listar lo creado, crear nuevo con selector de ejercicios del catálogo. Al eliminar rutina, confirmación en UI para no borrar por accidente. Si la lista está vacía, el mensaje anima a crear la primera.

Detalles de rutina / ejercicio — Donde muestro series, repeticiones, descansos y, si aplica, unidades inferidas para cardio continuo. En ejercicios con vídeo, el enlace debe abrirse en navegador o reproductor según implementación; comprobé que URLs vacías no rompan la UI.

Home / pantalla principal (si está separada de Main en el proyecto) — Suele mostrar resumen o accesos rápidos; dependiendo de la versión del código puede vivir como pestaña dentro de `MainScreen`. Si no la separé, es porque consolidé todo en main para reducir complejidad de navegación.

33. `AuthController` — detalles que me costaron entender al principio

Cada método (`registrar`, `login`, variantes Google/Firebase) tiene su propia matriz de errores: duplicados de email, oposición inválida, credenciales malas, usuario Firebase no registrado aún. Normalizo email en controlador antes de servicio para que **toda** la capa baje coherente.

Si `JWT_SECRET` falta, respondo 500 explícito: prefiero que falle en configuración que firmar con undefined.

34. Pool de MySQL (`config/db.js`)

Uso pool de conexiones en lugar de abrir una conexión nueva por petición. En servicios con transacción cojo `getConnection()` explícito y **siempre** `release()` en `finally`. Si olvidas `release`, el pool se agota y el servidor “deja de responder” de forma silenciosa tras un tiempo —error clásico que tuve en mente al escribir `ProgresoService`.

35. `ProgresoController` y parámetros de URL

Las rutas de evolución e historial llevan `userId` en path; el patrón correcto es el mismo que en rutinas: comparar con `req.usuario.id`. Si no lo hiciera, cualquiera autenticado podría pedir historial de otro — revisé el controlador para confirmar que aplica la misma filosofía defensiva.

36. `EjerciciosController` y catálogo

Listar ejercicios es la base de rutinas libres. Sin catálogo, el usuario no puede componer nada. Mantengo el catálogo en tabla dedicada para no duplicar nombres en cada rutina.

37. `OposicionController` y RSS

El controlador llama a `RssService` para agregar feeds. Si el servicio devuelve lista vacía, la UI puede mostrar un estado “sin noticias” sin crashear. Si devuelve error, decido si 500 o 200 con mensaje — depende de cómo lo implementé; lo importante es **no mezclar** fallo de RSS con fallo de datos estáticos de oposición.

38. Compose: estado y recomposición

Compose redibuja cuando cambia el estado observado. Si pongo lista de resultados en `mutableStateOf` y la actualizo tras la API, la lista se refresca sola. Si mezclo `remember` mal con datos que vienen de fuera, puedo ver UI “congelada” —cuando eso pasó, casi siempre era que el estado vivía en el sitio equivocado o que no propagaba error.

39. Ejemplo de flujo de datos (login, sin pegar respuestas kilométricas)

1. Usuario escribe email/password y pulsa entrar.
2. `ViewModel` llama `suspend fun login`.
3. Retrofit POST a `/api/auth/login`.
4. Servidor valida y responde con token + usuario.
5. `ViewModel` guarda token vía `TokenManager` y marca sesión.
6. Navegación a MAIN con stack limpio.

El “contrato” que memorizo es: **respuesta exitosa contiene token**; el resto de campos los miro en `AuthResponse.kt`.

40. Internacionalización (i18n) — lo que no hice y por qué

La app está en **español** de forma consistente en mensajes de error del servidor y textos de UI que yo controlé. Un paso natural sería `strings.xml` por idioma y mensajes localizados en cliente. No lo abordé en profundidad por tiempo; lo dejo como mejora si el proyecto continuara.

41. Copias de seguridad y datos

En desarrollo, mi “backup” es el `seed.sql` + exportaciones manuales de MySQL cuando hice cambios gordos. En producción real habría backups automáticos y snapshots; no forma parte del TFG pero sí del **discurso** si preguntan despliegue.

42. Control de versiones (Git)

Trabajé con ramas y commits razonables: features por bloques (auth, rutinas, historial). Cuando algo rompía la app, `git diff` y `git checkout` me salvaron. Recomendando a futuro yo mismo: commits pequeños con mensajes que expliquen **por qué**, no solo “cambios”.

43. Convenciones de nombres que intenté mantener

- Controladores en `*Controller.js`, servicios en `*Service.js`.
- Rutas en `*Route.js`.
- En Kotlin, pantallas `*Screen.kt`, view models `*ViewModel.kt`.
- Modelos de respuesta con sufijo `Response` o nombre explícito del recurso.

No siempre fui perfecto, pero cuando encontré inconsistencias, las corregí antes de que el proyecto creciera más.

44. Logging y depuración

`console.error` en controladores para excepciones inesperadas. En Android, `Log.d` / `Log.e` con tag coherente (paquete). Evité loguear tokens o passwords —una vez casi metí un log del token en desarrollo; lo quité en cuanto lo vi.

45. Glosario técnico rápido (mi uso)

- **JWT** — Token firmado que expira.
 - **ORM** — No usé ORM completo; SQL explícito con `mysql2`.
 - **DTO** — Mis `responsemodels` cumplen rol similar.
 - **Upsert** — Update o insert si no existe.
 - **Idempotencia** — No la garanticé en todos los endpoints; el historial diario es una forma de “casi idempotencia” funcional.
-

46. Preguntas que me hago al revisar el código antes de entregar

¿Hay algún `TODO` olvidado que rompa build? ¿Algún `http://` hardcodeado que deba ser configurable? ¿Algún `userId` del cliente que deba validarse contra token? ¿El `README` o `.env.example` reflejan las variables necesarias?

47. Cómo narrar el proyecto en la defensa oral (apuntes para mí)

Empiezo por el problema del usuario, luego arquitectura en tres capas, luego demo guiada. Si me piden detalle técnico, bajo a base de datos y JWT. Si me piden limitaciones, las digo sin titubear: TFG no es producto SaaS. Si me piden seguridad, hablo de bcrypt, JWT, verificación de tokens de terceros en servidor y comparación de ids de usuario.

48. Si tuviera que reimplementar solo una parte

Elegiría **historial + transacciones** porque es donde más duele un bug: datos huérfanos o duplicados. Una vez eso es sólido, el resto del sistema “descansa” sobre algo confiable.

48 bis. Segundo candidato a reimplementar con calma: cálculo de nivel

El cálculo de notas cruzando `baremos_puntuacion` y `mejor_si_es_menor` es delicado: un off-by-one en la comparación o un orden incorrecto en el `ORDER BY` puede asignar una nota demasiado optimista. Si tuviera que blindarlo, escribiría casos de prueba con tablas de baremo mínimas inventadas y marcas conocidas, y compararía resultados esperados vs obtenidos sin tocar aún la app.

48 ter. Tercer candidato: perfil y marcas (upsert + limpieza)

El controlador de perfil mezcla físico y marcas en un solo endpoint por comodidad de uso. Esa mezcla es potente pero más difícil de testear. Alternativas: separar endpoints o usar transacción única envolviendo todo. Lo dejo anotado porque si el proyecto creciera, separaría responsabilidades para aislar fallos.

49. Anexo: fallos típicos que me encontré y qué chequeé

“**La app no conecta**” — IP incorrecta, firewall, servidor caído, o URL base mal escrita en el cliente.

“**401 en todo**” — token caducado o cabecera sin `Bearer` .

“**403 al crear rutina**” — `userId` del body no coincide con token.

“**No me calcula el nivel**” — faltan marcas de alguna prueba oficial de la oposición; no es bug, es estado incompleto.

“**No puedo registrar dos veces el entreno**” — regla diaria; si quieres otro entreno, conceptualmente es otro día o otra rutina.

50. Anexo: cómo explico el JWT en una frase en la defensa

“El servidor firma un token cuando demuestras identidad; las rutas privadas verifican la firma con un secreto compartido; el token caduca para limitar el daño si alguien lo roba.” No hace falta matemáticas del algoritmo en la oral si dominas la idea de **firma** y **expiración**.

51. Anexo: bcrypt sin entrar en teoría de números

Guardar hash en lugar de contraseña implica que ni yo con acceso a base de datos puedo “ver” la contraseña original; solo puedo comparar un intento de login hashado con el hash guardado. El coste numérico de bcrypt ralentiza ataques de fuerza bruta masivos. Para el TFG es suficiente saber **por qué** está ahí.

52. Anexo: por qué el servidor agrega RSS y no el móvil directamente

Porque así controlo user-agent, timeouts, límites de items y un formato JSON estable para la app. Si cada cliente parseara RSS distinto, tendría inconsistencias y más superficie de error. Centralizar en servidor también facilita cache o políticas de uso futuras.

53. Anexo: qué significa para mí “código limpio” en este proyecto

No es que cada función tenga cinco líneas; es que cada capa tenga **una razón para cambiar**. Si cambia la normativa de baremos, toco servicios y quizá tablas, no la UI directamente. Si cambia la forma del botón, toco Compose, no el SQL. Cuando mezclé responsabilidades por prisa, después lo noté en los bugs.

54. Anexo: lecturas que me ayudaron (sin bibliografía formal)

Documentación de Android, Express, mysql2, y artículos sobre JWT y buenas prácticas REST. No copié un proyecto entero; fui leyendo trozos según me atascaba. Eso explica por qué algunas partes del código siguen el estilo “lo resolví cuando tocaba”.

55. Anexo: qué no repetiría

Dejar para el final la integración de terceros (Google/Firebase) suele doler; en el próximo proyecto similar integraría auth social antes, si el alcance lo incluye, para no acumular sorpresas de configuración al final del curso.

56. Mensaje final para el lector (yo del futuro)

No te castigues si no recuerdas cada endpoint: recuerda el **mapa**: tablas → servicios → rutas → pantallas. Ese mapa es lo que te hará rápido otra vez cuando el proyecto lleve meses archivado.

56 bis. Simulación mental de datos: un opositor ficticio

Imaginemos a **Usuario 42**, género hombre, oposición 1, con tres pruebas oficiales en esa oposición. Si solo tiene marcas registradas para dos de ellas, el cálculo de nivel debe decir “falta una prueba” y no inventar un nivel avanzado porque una de las dos marcas es buena. Eso es exactamente lo que codifiqué como `pruebasFaltantes`.

Cuando complete la tercera marca, el bucle de baremos recorre cada marca, busca la mejor nota compatible en `baremos_puntuacion` respetando `mejor_si_es_menor`, suma y promedia. Si una prueba no tiene baremo que encaje, el contador de pruebas usadas refleja la realidad —en un producto real auditaría esos huecos.

Con la nota media, asigno BASICO / INTERMEDIO / AVANZADO. Luego busco en `rutinas_opo` una fila que coincida en oposición, nivel, género y enfoque (el enfoque puede venir de la navegación del usuario en la app). Si no hay rutina para esa combinación, respondo 404 con mensaje comprensible: no es que el usuario “esté mal”, es que **falta contenido** en base de datos para esa combinación.

Este relato mental me sirvió más que cualquier diagrama UML para saber qué probar cuando cambiaba semillas SQL.

56 ter. Detalle de `obtenerRutinaCompleta` (idea general)

Sin citar cada línea, la función arma la rutina uniendo cabecera `rutinas_opo` con `detalle_rutina_opo` y `ejercicios` para obtener nombres y metadatos. El resultado agrupado es lo que consume la pantalla de detalle: lista ordenada de ejercicios con series, repeticiones, descansos. Si un ejercicio de cardio debe mostrarse como tiempo o distancia, la heurística de unidad entra en juego en el servicio o en la capa de presentación según lo implementé.

56 quater. Borrado de cuenta y efectos en cascada (qué debes revisar tú)

Si implementé borrado de usuario, hay que decidir política sobre historial, rutinas pers y marcas. Opciones: borrado en cascada configurado en BD, borrado explícito por servicio, o anonimización. Documento la pregunta porque es típica en defensa (“¿qué pasa con los datos personales?”). Mi implementación concreta está en `UsuarioController` / servicio asociado —léela antes de prometer GDPR perfecto.

56 quinquies. Rate limiting: qué protege y qué no

El limitador por IP frena abuso básico de endpoints. No detiene un atacante distribuido ni reemplaza WAF. Para el TFG es una capa razonable de **buena fe** y demuestra que pensé en abuso de login.

56 sexies. CORS y por qué parece “magia” hasta que falla

En desarrollo abrir CORS amplio evita dolores con pruebas desde herramientas web. En producción conviene restringir orígenes. Si un día expones un panel web al mismo API, CORS vuelve a ser protagonista.

56 septies. Versiones de dependencias: lección aprendida

Un día Gradle se quejó de incompatibilidad entre Kotlin y Compose Compiler. Solución: alinear versiones según tabla de compatibilidad oficial. Guardé esa experiencia como: **no actualizar todo a la vez sin leer notas de migración**.

56 octies. Qué guardo en `SessionManager` frente a `TokenManager`

El token es credencial; la “sesión” puede incluir flags de usuario o id cacheado. Prefiero no duplicar estado que pueda desincronizarse con `/me`. Si mezclé ambos, fue por pragmatismo; si unificara, usaría una sola fuente de verdad post-login.

56 nonies. Pruebas en Android: lo que no automatizé

No monté Espresso para cada pantalla; el tiempo iba a tests manuales y a priorizar backend middleware. En un trabajo futuro, al menos un test de humo de navegación login→main sería oro.

56 decies. Observabilidad: qué metería con más tiempo

Request ids, logs estructurados, métricas de latencia por endpoint. Aquí tengo `console.error` y logs de Android. Suficiente para TFG; insuficiente para producción seria.

56 undecies. Docker (no usado) y por qué lo menciono

Podría empaquetar MySQL + Node en `docker-compose` para reproducibilidad total. No lo hice en todos los entornos; si el tribunal pregunta, la respuesta honesta es “prioricé entrega funcional”; la mejora es clara.

56 duodecies. HTTPS en producción

En local uso HTTP; en internet real TLS obligatorio. El despliegue con Let's Encrypt o proxy reverso no está en el alcance; lo nomino para no confundir seguridad de aplicación con seguridad de transporte.

56 terdecies. Separación de entornos `dev` / `prod`

Idealmente variables distintas y bases distintas. En TFG suele haber un solo `.env` local. Si continúo el proyecto, separaría secretos y activaría configuraciones por `NODE_ENV`.

56 quaterdecies. Cómo hablar de Firebase sin sonar a marketing

Firebase Auth resuelve identidad del lado cliente; mi servidor **no confía** ciegamente: verifica el token con Admin SDK. Esa frase es la que importa técnicamente.

56 quinquedecies. Cómo hablar de Google Sign-In

Similar: ID token verificado contra `GOOGLE_CLIENT_ID`. El usuario ve un botón bonito; yo veo criptografía y validación de audiencia.

56 sexdecies. Mensajes de error en español

Los puse en servidor para que cliente y usuario vean lo mismo sin traducciones duplicadas mal sincronizadas. Si un día internacionalizo, quizá pase a códigos de error estables + catálogo i18n en cliente.

56 septdecies. Qué haría con PII en una versión 2.0

Minimizar datos, política de retención, exportación y borrado granular. El TFG toca la superficie; el RGPD completo es proyecto aparte.

56 octodecies. Firma de commits y trazabilidad

Si el tribunal valora procedencia, commits claros ayudan a demostrar autoría progresiva. Yo intenté que cada feature mayor tuviera commit identificable.

56 novodecies. README del backend

Debería listar: requisitos, cómo crear `.env`, cómo migrar BD, cómo arrancar. Si falta algo, completarlo antes de entregar —es la primera puerta para quien evalúe sin mi presencia.

56 vicesim. Última nota antes del cierre

Si algo de este documento contradice el código, **gana el código** tras actualizarlo; este texto es la narrativa que yo necesitaba, no un contrato legal. Pero si el código tiene un bug y el documento describe la intención correcta, el bug sigue siendo bug —el documento entonces sirve para saber qué **debía** pasar.

57. Guía de reimplementación por fases (el plan que seguiría yo otra vez)

Fase A — Datos. Lo primero que haría otra vez es **pararme a dibujar** el diagrama entidad-relación en papel o pizarra, sin abrir el IDE. Cuando salté directo al código en algún experimento fallido, acabé con tablas que no encajaban con las pantallas. Tras el dibujo, crearía el `CREATE TABLE` en `mydb.sql` respetando tipos ENUM donde el dominio es cerrado y DECIMAL donde hay decimales reales. Cargaría semillas mínimas con **dos oposiciones** para probar que el usuario puede elegir; pruebas oficiales asociadas; baremos con varias filas por prueba para ver el comportamiento del join; ejercicios suficientes para montar al menos una rutina opo con tres líneas de detalle y otra rutina pers de ejemplo. Insertaría datos a mano vía cliente SQL para validar FK antes de confiar en el backend.

Fase B — Servidor mínimo. Montaría `app.js` con `dotenv`, `cors`, `express.json`, y una ruta trivial `GET /health` que devuelva `{ ok: true }` para confirmar que el proceso arranca. Conectaría el pool de MySQL y haría `SELECT 1` o leería `oposiciones` limitado. Hasta que esto no funcione, no añado autenticación: aislar fallos de red y credenciales es más rápido sin capas extra.

Fase C — Auth básico. Implementaría `AuthService.registrar` y `login` con transacción en registro solo donde haga falta, `bcrypt` con coste razonable, y JWT firmado con secreto largo. El middleware `validarToken` lo probaría con Postman: primero `login`, copiar token, llamar a `/api/auth/me` o una ruta dummy protegida. Si algo falla, revisaría `Authorization: Bearer` con espacio correcto —un error tonto que puede perder media hora.

Fase D — Dominio oposiciones. Endpoints de listado y detalle con SQL legible. Pospondría RSS si me bloquea: el núcleo deportivo no depende de noticias externas. Si el listado público de oposiciones es necesario para el registro, lo dejaría antes de exigir token en todo.

Fase E — Rutinas oficiales. Aquí invertiría el mayor tiempo de pruebas unitarias manuales: `calcularNotaYNivel` con marcas incompletas, marcas completas, baremos vacíos, y `obtenerRutinaCompleta` con múltiples `enfoque_tipo`. Validaría el JSON frente a lo que la app espera en `DetallesRutinaScreen` antes de pintar UI.

Fase F — Rutinas pers y ejercicios. Catálogo primero; luego creación validando que el `userId` del cuerpo coincide con el token. Probaría 403 forzando otro id. El listado agrupado lo validaría con dos rutinas y ejercicios repetidos para ver que el `Map` en controlador no mezcla bloques.

Fase G — Historial transaccional. Escribiría primero el caso feliz con transacción; luego el duplicado diario; luego un fallo a mitad de inserción de resultados para confirmar rollback. Revisaría que `connection.release()` siempre ocurre.

Fase H — Info pruebas. Joins de baremos y marcas; comprobaría que `getMarcasUsuario` filtra por oposición correcta y que las unidades mostradas tienen sentido para el tribunal.

Fase I — RSS. Integraría feeds con timeouts, límites y respuestas vacías tolerantes. Probaría con `RSS_DEBUG` y sin red para ver el comportamiento.

Fase J — Android esqueleto. Proyecto Compose con navegación mínima, `TokenManager`, y una sola llamada autenticada exitosa antes de construir pantallas complejas.

Fase K — Pantallas de dominio. Una pantalla a la vez: entrenamientos cuando rutinas oficiales funcionan; historial cuando registro funciona. Evito el “todo a la vez” que compila pero no demuestra nada.

Fase L — Pulido UX. Estados vacíos, spinners, mensajes de error; borrar cuenta de prueba y re-registrar para ver flujo limpio.

Fase M — Terceros. Google/Firebase cuando el núcleo ya demuestra valor; si falla la config, no bloquea lo demás.

Ese orden es **personal**; lo documento para que el “yo del futuro” no empiece por Compose bonito y termine sin tablas.

57.1. Si tuviera que explicar el proyecto en veinte minutos (guion)

Minutos 1–3: problema del opositor y qué aporta OpoFit. 4–8: arquitectura tres capas y por qué. 9–14: demo login → rutina → registrar → historial. 15–18: seguridad básica y decisiones de datos. 19–20: límites y trabajo futuro. Sin vivir del código línea a línea salvo que pregunten.

57.2. Cosas que deliberadamente no automatizo en este documento

No genero OpenAPI automáticamente desde aquí; el código y los modelos Kotlin son la fuente. No duplico el contenido de `mydb.sql` entero: está en el repo. No listo cada propiedad de cada data class: cambia con el tiempo.

57.3. Cómo sé que esta documentación “terminó”

Termina cuando cubre **arquitectura, datos, auth, rutinas, historial, cliente Android, flujos, fallos, despliegue local y plan de reimplementación**. Lo demás es bonus.

58. Cierre de esta documentación

Si lees esto junto al código, deberías poder **seguir el hilo** de cualquier funcionalidad: qué tabla toca, qué servicio manda, qué pantalla dispara la acción. Eso es exactamente lo que yo necesitaba cuando unía piezas a altas horas.

Cuando algo no cuadre, **la fuente de verdad es el repositorio**; este documento es el mapa que escribí para no perderme yo primero.

Un último apunte: la documentación larga no sustituye **probar** el sistema. Incluso con todo esto escrito, si no abro la app y no registro un entreno de punta a punta cada vez que toco historial, puedo romper lo que creía estable. Por eso el ciclo lectura → código → prueba manual sigue siendo mi ritual.

Fin de la documentación técnica personal — OpoFit.